

CPP3-20 Final Report: Fault Tolerant, Distributed In-Memory KV Store

Team: CPP3-20

Project: Fault Tolerant, Distributed In-Memory KV Store

Components: Consistent Hashing, Buddy Replication, Gossip Membership, RPC Layer, Shared-Memory Storage, Benchmarking

Status: Complete

Github Repository: <https://github.com/OneRandom1509/HPE-distributed-key-value-store>

Team Members:

1. Ananta Krishnan B
2. Samar Singh
3. Kanad Bhattacharya
4. Priyanshu Kumar Jha
5. Mishthi Jain

Table of Contents

1. [Abstract](#)
2. [System Architecture](#)
3. [Storage Layer](#)
4. [RPC Layer](#)
5. [Consistent Hashing and Key Distribution](#)
6. [Buddy Replication](#)
7. [Gossip Membership Protocol](#)
8. [Client CLI and Interactive Interface](#)
9. [Benchmarking](#)
10. [Configuration](#)
11. [Build System and Dependencies](#)
12. [Deployment](#)
13. [Problems Faced](#)
14. [Failure Scenarios and Handling](#)
15. [Future Improvements](#)

1. Abstract

This project implements a distributed, fault-tolerant key-value store that runs across a cluster of nodes. Keys are distributed across nodes using a **consistent hash ring** with **virtual nodes** rather than a naive modulo-based approach. Each physical node is represented by their virtual node positions evenly spaced across the 32-bit ring, ensuring balanced load distribution without manual token management. Each key is replicated to a **buddy node** so that a single node failure does not cause data loss. **Lazy migration** automatically repopulates a recovered primary node when a read triggers a buddy fallback, eliminating cold-start gaps after node failures. A **gossip-based membership protocol** allows the cluster to automatically detect node joins, departures, and failures, and propagate topology changes to all clients.

The system uses **Thallium** (a C++ RPC framework built on Margo and Argobots) for network communication and **Boost.Interprocess** for shared-memory storage with optional persistence via memory-mapped files. A benchmarking subsystem measures the remote operation latencies with both sequential and random access patterns.

2. System Architecture

2.1 Component Overview

Component	Files	Responsibility
KvStore	src/KVStore.cpp , include/KVStore.hpp	Shared-memory OR persistent storage; singleton CRUD; interprocess mutex synchronization
KVServer	src/KVServer.cpp , include/KVServer.hpp	Thallium RPC provider; registers 6 RPC handlers
KVClient	src/KVClient.cpp , include/KVClient.hpp	Thallium RPC caller; timed async RPC; per-instance engine
KVDistributor	src/KVDistributor.cpp , include/KVDistributor.hpp	Key routing logic; primary/buddy dispatch; ring rebuild from membership
ConsistentHashRing	src/ConsistentHashRing.cpp , include/ConsistentHashRing.hpp	Ring construction; primary/buddy lookup; binary search; range assignment
GossipMembership	src/GossipMembership.cpp (390 lines), include/GossipMembership.hpp	Heartbeat, failure detection (SUSPECT/DEAD), peer

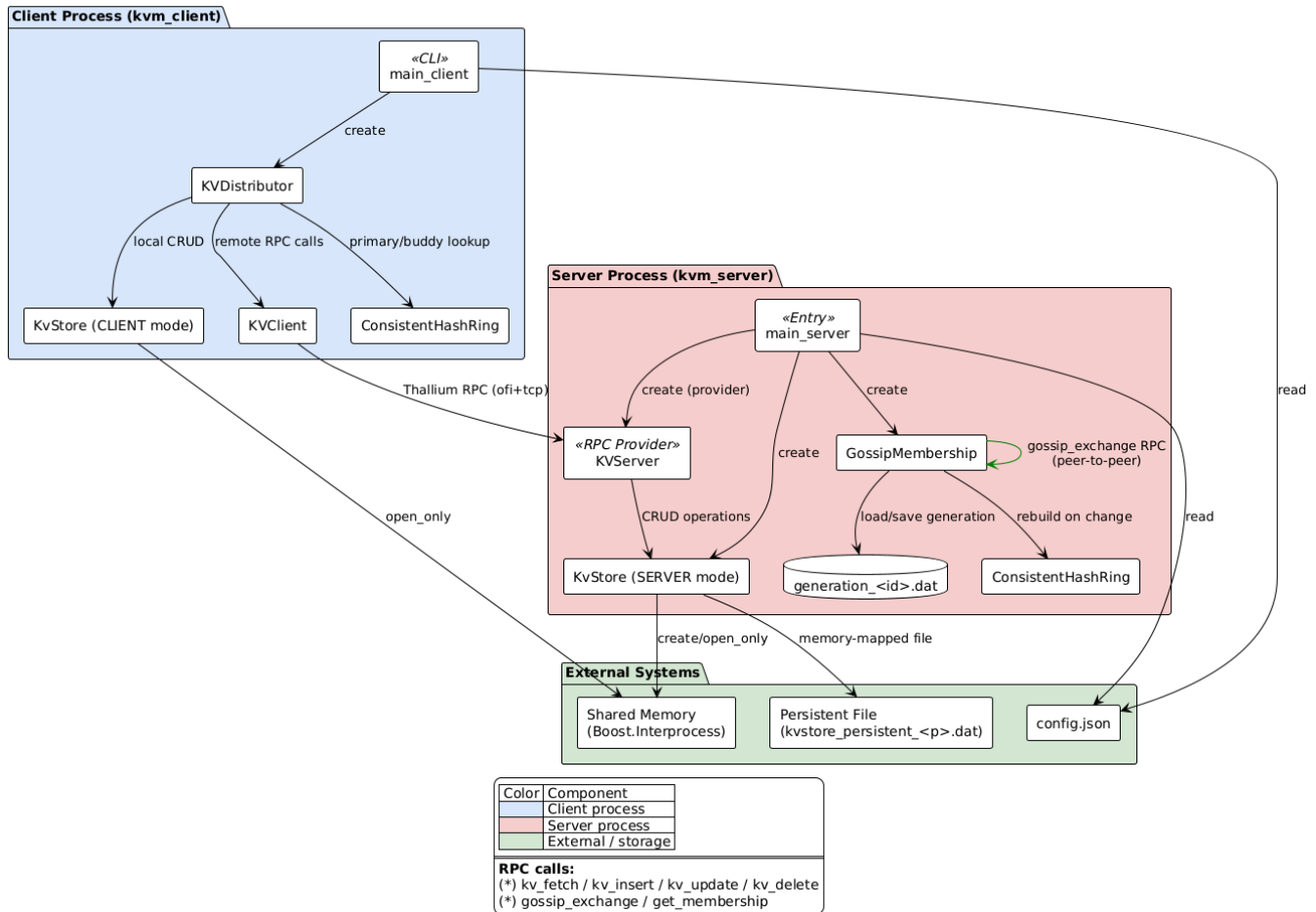
Component	Files	Responsibility
		exchange, change callback
MurmurHash3	src/MurmurHash3.cpp , include/MurmurHash3.hpp	32-bit MurmurHash3 for key and token hashing
Config	src/config.cpp , include/config.hpp	JSON config parser; protocol, endpoints, gossip settings
Timer	src/Timer.cpp , include/Timer.hpp	High-resolution stopwatch for RPC timing and benchmarks
main_server.cpp	src/main_server.cpp	Server entry point; engine init, gossip bootstrap, signal handling
main_client.cpp	src/main_client.cpp	Client CLI entry point; interactive REPL; benchmarks

2.2 Process Model

The system consists of two executables:

- **kvm_server** : One instance per cluster node. Owns a Thallium engine in server mode, a KvStore instance with per-node shared memory, a GossipMembership instance that runs a background gossip thread, and a KVServer provider that handles incoming RPCs.
- **kvm_client** : One or more client instances. Owns a KVDistributor with a KVClient (Thallium engine in client mode) for RPC communication. Periodically refreshes membership from any reachable server endpoint to keep its hash ring up to date.

2.3 Architecture Diagram



3. Storage Layer

3.1 Singleton Pattern

KvStore is a per-process singleton accessed via:

```
KvStore &store = KvStore::get_instance(size, mode, conn_mode, node_tag);
```

The `conn_mode` parameter distinguishes between **SERVER** (creates new shared memory) and **CLIENT** (attaches to existing shared memory). The `node_tag` is appended to shared-memory segment names so that each node has its own isolated storage even when running on the same machine.

3.2 Storage Modes

Memory mode uses `boost::interprocess::managed_shared_memory` to allocate a fixed-size region. A `boost::unordered_map` is constructed inside the shared memory using custom allocators (`MemoryMapAllocator`). This allows the client process to directly read from the same memory region the server writes to, in essence a zero-copy local access.

Persistent mode uses `boost::interprocess::managed_mapped_file` which maps a file on disk (`kvstore_persistent_<port>.dat`) into the process address space. Data survives server restarts. The storage API is identical to memory mode; only the backing segment manager differs.

3.3 Thread Safety

All CRUD operations (`Insert` , `Update` , `Delete` , `Find`) acquire a `boost::interprocess::named_mutex` (unique per node) before accessing the shared map. The mutex name is derived from the node tag to prevent cross-node interference.

3.4 CRUD Operations

Operation	Signature	Behavior
Insert	<code>Insert(int key, const string &value)</code>	Checks for duplicate; allocates in shared memory
Find	<code>Find(int key) -> string</code>	Returns value or "key not found"
Update	<code>Update(int key, const string &new_value)</code>	Replaces existing value; errors if key missing
Delete	<code>Delete(int key)</code>	Removes key; reports freed bytes
Sync	<code>Sync()</code>	Flushes mapped file to disk (persistent mode only)
ListAllKeys	<code>ListAllKeys()</code>	Debug: prints all keys in the map

The `Find` operation distinguishes three cases for RPC routing:

- "key not found" : key does not exist locally (trigger remote fallback)
- "Map not found" : storage segment not initialized
- Actual value string: successful local read

3.5 Memory Statistics

`GetMemoryStats()` returns `MemoryStats{total_size, used_memory, free_memory, usage_percent}` . The servers print these at after every operation to show the shared memory statistics.:

```
===== MEMORY STATS [After <operation>] =====
Storage mode: MEMORY
Total memory: 100.00 MB
Used memory:  0.00 MB
```

```
Free memory: 100.00 MB
Usage:       0.00%
```

4. RPC Layer

4.1 Thallium Engine

Both server and client use **Thallium** (mochi-thallium), a C++ wrapper around the Margo RPC runtime. Margo runs on top of Argobots (lightweight user-level threading) and Mercury (network transport abstraction). The transport protocol is `ofi+tcp` (libfabric over TCP).

- **Server** creates one engine: `tl::engine myEngine(address, THALLIUM_SERVER_MODE)`
- **Client** creates one engine per `KVClient` instance: `tl::engine myEngine(protocol, THALLIUM_CLIENT_MODE)`

4.2 KVServer: RPC Provider

`KVServer` extends `tl::provider<KVServer>` and registers six RPC handlers in its constructor:

```
define("kv_fetch",      &KVServer::kv_fetch);
define("kv_insert",     &KVServer::kv_insert);
define("kv_update",     &KVServer::kv_update);
define("kv_delete",     &KVServer::kv_delete);
define("gossip_exchange", &KVServer::gossip_exchange);
define("get_membership", &KVServer::get_membership);
```

Handler	Args	Returns	Description
<code>kv_fetch</code>	<code>int key</code>	<code>string value</code>	Looks up key in <code>KvStore</code>
<code>kv_insert</code>	<code>int key, string value</code>	<code>int (1=ok, 0=fail)</code>	Inserts key-value pair
<code>kv_update</code>	<code>int key, string value</code>	<code>int (1=ok, 0=fail)</code>	Updates existing key
<code>kv_delete</code>	<code>int key</code>	<code>int (1=ok, 0=fail)</code>	Deletes key
<code>gossip_exchange</code>	<code>vector<MemberRecord></code>	<code>vector<MemberRecord></code>	Merges gossip view and returns local view
<code>get_membership</code>	<code>none</code>	<code>vector<MemberRecord></code>	Returns current member table

Handler	Args	Returns	Description
			snapshot

Each handler uses a `Timer` to log operation latency on the server side.

4.3 KVClient: RPC Caller

`KVClient` wraps each Thallium RPC call with optional timeout support. The pattern for each RPC:

```
tl::remote_procedure remote_rpc = myEngine.define("rpc_name");
tl::endpoint server_ep = myEngine.lookup(protocol + "://" + endpoint);
tl::provider_handle ph(server_ep, provider_id);
auto req = remote_rpc.on(ph).timed_async(timeout, args...);
result = req.wait().as<ReturnType>();
```

If `rpc_timeout_ms > 0`, timed async is used; otherwise synchronous calls are made. Timeout support prevents the client from hanging indefinitely when a server is unreachable.

5. Consistent Hashing and Key Distribution

5.1 Hash Ring

The ring uses a 32-bit circular address space `[0, 232 - 1]`. Both keys and node tokens are hashed onto this ring using MurmurHash3 (32-bit variant). The ring is stored as a `std::vector<Token>` sorted by position:

```
struct Token {
    uint32_t position; // ring position [0, 2^32)
    int node_id;       // physical node owner
};
```

Lookup uses `std::lower_bound` (binary search, $O(\log N \cdot V)$).

5.2 Virtual Nodes

Each physical node occupies 150 virtual node positions on the ring by default. Virtual nodes are evenly spaced by dividing the ring into equal intervals:

```
const uint64_t interval = (1ULL << 32) / virtual_nodes_per_node_;
for each virtual node v:
    position = hash(node_id) + v * interval
```

This ensures balanced load distribution without the clustering that would occur with purely random token placement.

Note: For the sake of demonstration and proof of concept, each physical node will only have 2 virtual nodes associated with it.

5.3 Primary and Buddy Lookup

Primary node: walk clockwise from the key's hash position to the first token.

```
int getPrimaryNode(int key) {
    auto it = lower_bound(ring_, hashKey(key));
    if (it == ring_.end()) it = ring_.begin(); // wrap
    return it->node_id;
}
```

Buddy node: continue clockwise from the primary's token until a token belonging to a different physical node is found.

```
int getBuddyNode(int key) {
    auto start = findToken(hashKey(key));
    auto it = start;
    do {
        ++it; if (it == ring_.end()) it = ring_.begin();
        if (it->node_id != start->node_id) return it->node_id;
    } while (it != start);
    return start->node_id; // single-node fallback
}
```

5.4 KVDistributor Routing

`KVDistributor` is the central routing class used by the client. It holds a `ConsistentHashRing`, a `KVClient` for remote RPCs, and a reference to the local `KvStore`.

Read path (get):

1. Compute `primary_id = ring.getPrimaryNode(key)`
2. If primary is local → `kv.Find(key)`
3. Else → `kv_client->fetch(key, endpoint)`
4. If primary fails → fall back to buddy (same logic)
5. If buddy has the key and `buddy_id != primary_id` → **lazy migrate**: write the key-value pair back to the primary via `writeToNode(primary_id, key, value, RemoteOp::INSERT)`, so subsequent reads hit the primary directly

6. If both fail → return "RPC Failed"

Write path (insert , update , deleteKey):

1. Compute primary_id and buddy_id
2. Write to primary (local or RPC)
3. If buddy_id != primary_id , also write to buddy
4. Catch exceptions per-node to avoid one failure blocking the other

6. Buddy Replication

6.1 Write Path

Every mutating operation writes to **both** the primary and the buddy node synchronously. The primary write is considered mandatory; the buddy write is best-effort (logged on failure but does not block the operation).

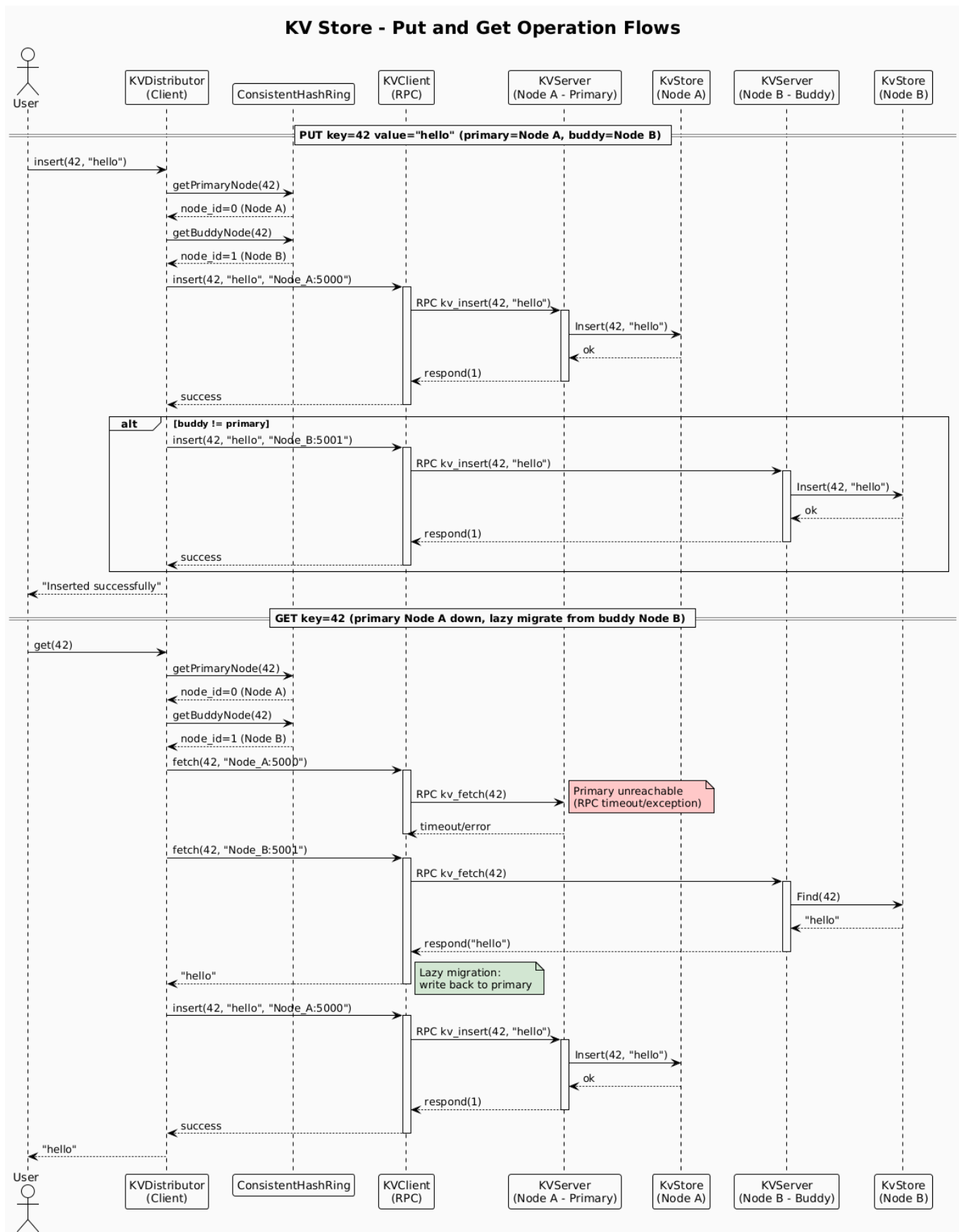
6.2 Read Path

Reads query the primary first. If the primary is unreachable or returns "key not found" , the read falls back to the buddy. This provides fault tolerance: the system remains readable even when one of the two replica holders is down.

6.3 Self-Write Deduplication

In a single-node cluster, getBuddyNode returns the same node as getPrimaryNode . The buddy_id != primary_id check prevents issuing the same write twice to the same node.

6.4 Sequence Diagram



7. Gossip Membership Protocol

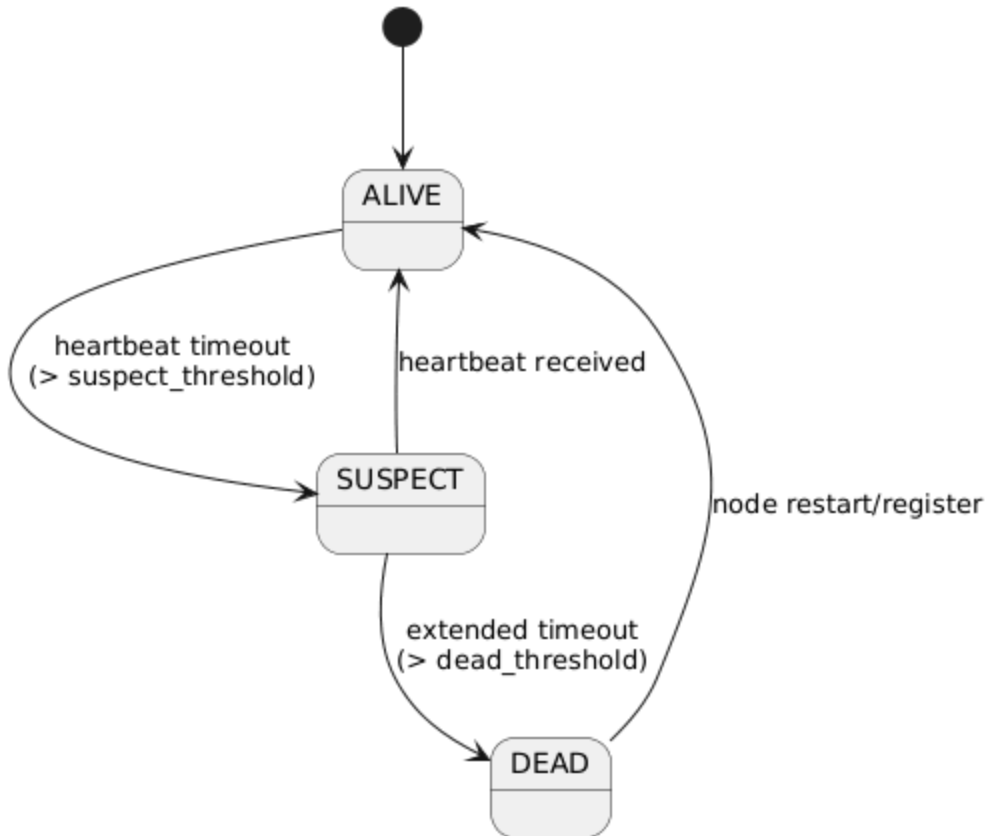
7.1 MemberRecord Structure

Each member is tracked with:

```
struct MemberRecord {  
    int node_id;  
    std::string endpoint;           // "192.168.1.48:5000"  
    NodeStatus status;             // ALIVE=0, SUSPECT=1, DEAD=2  
    uint64_t heartbeat_counter;    // monotonically increasing  
    uint32_t generation;           // incremented on each restart  
};
```

The `generation` field is persisted to a file (`generation_<node_id>.dat`) and incremented on each server start. This prevents stale heartbeats from a previous incarnation of a node being mistaken for fresh ones.

7.2 State Machine



- `suspect_threshold_ms` : 3000 ms (configurable)
- `dead_threshold_ms` : 10000 ms (configurable)
- Gossip interval: 1000 ms (configurable)

7.3 Gossip Exchange

Each server runs a background gossip thread that executes the following at every interval:

1. Increment its own heartbeat counter.
2. Run failure detection (ALIVE → SUSPECT → DEAD transitions).
3. Pick a random ALIVE peer.
4. Send its full member table via the `gossip_exchange` RPC.
5. Merge the peer's response into its local table.

Merge rules (in `mergeView`):

- Unknown nodes are added.
- Higher generation → newer version wins (node restarted).
- Same generation, higher heartbeat → update.
- Same generation and heartbeat but more severe status → propagate.
- Self entry is never overwritten by a peer.

7.4 Change Notification

When the live set changes (a node transitions state or a new node is discovered), the `GossipMembership` checks whether the set of ALIVE nodes actually differs from the last known set. If so, it invokes the `MembershipChangeCallback`:

```
std::function<void(const std::unordered_map<int, std::string>&)>
```

On the server side, this callback rebuilds the server's local `ConsistentHashRing` and logs the new assignments. On the client side, the `get_membership` RPC is called periodically to poll the latest membership view, and `updateRingFromMembers` triggers a ring rebuild when the live set changes.

7.5 Seed Nodes

On startup, the server contacts its configured seeds (from `config.json`) to bootstrap its initial membership view. After this bootstrapping phase, gossip operates entirely peer-to-peer.

7.6 Clean Shutdown

When a server receives SIGINT/SIGTERM:

1. It marks itself DEAD in the member table.
2. It performs one final gossip round to announce departure.
3. It stops the gossip thread and finalizes the Thallium engine.

8. Client CLI and Interactive Interface

The client (`main_client.cpp`) provides an interactive REPL with the following commands:

Command	Syntax	Description
put	put <key> <value>	Inserts or updates a key-value pair
get	get <key>	Retrieves the value for a key
update	update <key> <value>	Updates an existing key's value
delete	delete <key>	Deletes a key-value pair
benchmark	benchmark	Runs sequential fetch benchmark (10k ops)
benchmark1	benchmark1	Runs random fetch benchmark (10k ops)

Key values are displayed in hexadecimal format:

```
> put 42 hello
Inserted on the server successfully: 42 -> hello

> get 42
Key not found locally. Fetching from server.
Key Found on the Server.
Inserted on the server successfully: 42 -> hello
Value: hello
```

9. Benchmarking

9.1 Benchmark Infrastructure

The `Timer` class uses `std::chrono::high_resolution_clock` for sub-millisecond precision:

```
class Timer {
    void start();           // records start timepoint
    void stop();            // records end timepoint
    double elapsed_ms();    // returns duration in ms
    void print(label);      // prints "[label] completed in X.XX ms"
};
```

9.2 Sequential Benchmark (`benchmark`)

1. **Insertion phase:** Insert 10,000 key-value pairs with random 3-character values. Keys are sequential integers 1–10,000.
2. **Fetch phase:** Fetch all 10,000 keys in sequential order (1, 2, 3, ..., 10000).

Output includes:

- Total and average insertion time
- Total, average, min, and max fetch time
- Success rate (successful fetches / total)

9.3 Random Benchmark (benchmark1)

Same insertion phase (10,000 sequential inserts), but the fetch phase uses a uniform random distribution over the key range `[1, 10000]`. This models a more realistic workload where access patterns are not predictable. The consistent hash ring handles routing transparently: all benchmarks go through the full `KVDistributor` dispatch path (local store or RPC), so the metrics reflect real-world operation cost regardless of which node owns the key.

9.4 Timer Usage in RPC Calls

Every RPC handler in `KVServer` wraps its operation in a `Timer` and logs the latency:

```
[Insert] completed in 0.682 ms
[Fetch] completed in 0.124 ms
```

This provides real-time observability into server-side operation costs.

10. Configuration

10.1 config.json

```
{
  "provider_id": 1,
  "protocol": "ofi+tcp",
  "count_of_node": 4,
  "ip_addresses": {
    "0": "172.16.10.122:5000",
    "1": "172.16.10.123:5000",
    "2": "172.16.10.124:5000",
    "3": "172.16.10.125:5000"
  },
  "size": 1,
  "local_ip": "",
  "gossip": {
    "interval_ms": 1000,
    "suspect_threshold_ms": 3000,
    "dead_threshold_ms": 10000,
    "seed_nodes": [
```

```

        "ofi+tcp://172.16.10.122:5000",
        "ofi+tcp://172.16.10.123:5000"
    ]
}
}

```

Field	Type	Description
provider_id	int	Thallium RPC provider ID (default: 1)
protocol	string	OFI transport protocol (e.g. ofi+tcp)
count_of_node	int	Total configured nodes
ip_addresses	object	Node ID → ip:port mapping
size	int	Shared memory size in MB
local_ip	string	Override for auto-detected IP (empty = auto)
gossip.interval_ms	int	Gossip exchange interval (ms)
gossip.suspect_threshold_ms	int	No-heartbeat time before SUSPECT (ms)
gossip.dead_threshold_ms	int	SUSPECT time before DEAD (ms)
gossip.seed_nodes	string[]	Initial gossip contact endpoints

11. Build System and Dependencies

11.1 Dependencies

All installed via Spack (detailed installation instructions are present in the repository's [README](#)):

Library	Role
mochi-margo	RPC runtime (Margo)
mochi-thallium	C++ RPC wrapper
argobots	Lightweight user-level threading
mercury	Network abstraction layer
boost	Interprocess shared memory, unordered_map
nlohmann-json	JSON configuration parsing
spdlog	Structured logging
GTest	Unit testing

11.2 CMake Targets

Target	Type	Source Files
kvm_server	Executable	8 source files
kvm_client	Executable	8 source files
consistent_hash_ring_tests	Test binary	Ring unit tests (custom harness)
kv_distributor_tests	Test binary	Distributor GTest suite

11.3 Makefile Targets

Target	Description
make build	Release CMake configure + build
make debug	Debug CMake configure + build
make server	Runs build/kvm_server
make client	Runs build/kvm_client
make nodes	Runs build/start_nodes.sh
make test	Runs all CTest tests
make test-kv	Runs kv_distributor_tests directly
make test-ring	Runs consistent_hash_ring_tests directly

12. Deployment

12.1 Server Startup

```
make server ARGS="[protocol] [port] [mem_size][K|M|G] [persistent|memory]"
```

The server auto-detects its IP via `hostname -I`, matches itself against `config.json` by IP:port to determine its `node_id`, initializes the Thallium engine, creates the shared-memory store, and starts the gossip membership protocol.

12.2 Client Startup

```
make client
```

The client prompts for storage mode (memory/persistent), connects to the existing shared memory created by the server, initializes a `KVClient` RPC engine, and enters the interactive CLI loop.

13. Challenges Faced

13.1 Argobots Scheduling Conflict with Multiple Thallium Engines

Symptom: The client crashed during startup with:

```
[error] Failed to retrieve main scheduler from ES
(ABT_xstream_get_main_sched returned 4) in
__margo_abt_xstream_init_external
terminate called after throwing an instance of
'thallium::margo_exception'
what(): [margo_init_ext] Could not initialize Margo
```

Root cause: The client initially created two `tl::engine` instances on different threads. The main thread created one engine via `KVClient` (inside `KVDistributor`), and a background membership refresh thread created a second engine using a separate `tl::engine(membership_protocol, THALLIUM_CLIENT_MODE)`. The second `margo_init_ext` call from the background thread invoked `ABT_xstream_get_main_sched`, which returned `ABT_ERR_INV_XSTREAM` (error code 4, "invalid execution stream"). Argobots associates its primary scheduler with the thread that first initialized Margo, and subsequent initialization attempts from other threads cannot retrieve it.

Fix: The background thread's standalone engine and raw RPC calls were replaced with calls to the existing `KVDistributor::fetchMembershipFromServer()` method, which reuses the `KVClient` engine already initialized on the main thread. This eliminated the redundant engine creation entirely.

13.2 Per-Node Storage Isolation

When running multiple server instances on the same machine (for testing a multi-node cluster), all nodes initially tried to create/connect to the same shared memory segment name. This was fixed by appending the port number as a `node_tag` to the shared memory name, mutex name, and persistent file path, creating per-node storage isolation.

14. Failure Scenarios and Handling

14.1 Primary Node Down on Read

`KVDistributor::get` catches the RPC exception, logs a warning, and falls back to the buddy node. If the buddy has the key (from buddy replication), the read succeeds and **lazy migration** automatically repopulates the primary, restoring full redundancy for that key.

14.2 Primary Node Down on Write

The primary write throws. The buddy write still proceeds. The data survives on the buddy. When the primary recovers, it will be discovered via gossip, and the client will serve reads from the buddy until gossip convergence triggers a ring rebuild.

14.3 Buddy Node Down on Write

The primary write succeeds. The buddy write throws and is logged as a warning. The key exists on the primary only until the buddy recovers. No data loss.

14.4 Both Primary and Buddy Down

Both RPC calls fail. Returns "RPC Failed". In persistent mode, data will be available when the nodes restart.

14.5 Single-Node Cluster

`getBuddyNode` returns the same node as `getPrimaryNode`. The deduplication check prevents double-writing. The system operates as a non-replicated single-node store.

14.6 Hash Collision

Virtual node positions are deterministic (base hash + evenly spaced intervals). The probability of two tokens sharing the same position with different node IDs is negligible, and `stable_sort` with a tiebreaker comparator (`node_id`) ensures deterministic ordering.

15. Future Improvements

- **Hinted handoff:** When a primary is unreachable, have the buddy store hinted writes and replay them when the primary recovers.
- **Phi Accrual Failure Detector:** Replace fixed timeouts with adaptive failure detection based on historical heartbeat intervals.
- **Multi-key operations:** Support atomic operations across keys that hash to the same node.
- **HTTP Gateway:** An HTTP proxy can be integrated with an implementation of KVClient for more abstraction and to ensure confidentiality and security of the internal network. A simple HTTP client can be used to perform CRUD operations on the system. A crude implementation of this can be seen in the branch [feat/http-gateway-impl](#) of the repository.